

Introduction to programming

Labor06

Revision

- ▶ GCD, LCM, Relative primes
- ▶ Random numbers
 - `random()`
 - `randint()`
- ▶ Lists
 - Slicing
 - Operators
 - Functions
- ▶ List Comprehension

Homework 1 – Neanderthal gene

<https://progcont.hu/progcont/100350/?pid=201515>

- ▶ A recent study suggests that the risk of coronavirus infection is more severe in people who carry a small Neanderthal gene. Reading this makes you stop for a moment and wonder if you have even an ounce of the Neanderthal gene in you.
- ▶ If you make a model, Neanderthal genes are relatively easy to identify. Suppose we describe a **DNA molecule** by a sequence of integers. **If we can find at least three consecutive values in this sequence that form a strictly monotonic series, we can say that the DNA molecule contains a Neanderthal gene.**
- ▶ Write a program that reads at least one positive integer per line from the standard input to the end of file (EOF). Each line describes a DNA molecule. If the DNA molecule under test contains a Neanderthal gene, your program should write a line containing a "YES" string to the standard output, otherwise a "NO" string.

Homework 1 – Neanderthal gene

<https://progcont.hu/progcont/100350/?pid=201515>

Sample Input

```
5 6 7
9 8 7 6 5
2 3
7 11 8 10 9
4 4 4 4
9
1 2 3 1 2
7 8 7 8 7 8 9
```

Output for Sample Input

```
YES
YES
NO
NO
NO
NO
YES
YES
```

Solution – Neanderthal gene

```
while True:
    try:
        numbers = input().split()
        n = len(numbers)
        DNS = False
        for i in range(n - 2):
            if int(numbers[i]) < int(numbers[i + 1]) < int(numbers[i + 2]) or
               int(numbers[i]) > int(numbers[i + 1]) > int(numbers[i + 2]):
                DNS = True
        if DNS:
            print("YES")
        else:
            print("NO")
    except EOFError:
        break
```

Homework 2 – Piggy bank

<https://progcont.hu/progcont/100353/?pid=201527>

- ▶ Children love to collect their savings in piggy banks. In this exercise, you have to make a statement of these savings.
- ▶ Write a program that reads the names of the children from standard input to end-of-file (EOF) and the money they have saved from week to week, line by line!
- ▶ The form of a line is:
 - ▶ `name:quantity[ quantity]...`
- ▶ Your program will write in the first line of the standard output how much money the children have saved in total according to the statement! While processing the data, also count the number of children who regularly saved at least 20 forints in their piggy bank every time! Print the latter value on the second line of the standard output!

Homework 2 – Piggy bank

<https://progcont.hu/progcont/100353/?pid=201527>

Sample Input

- ▶ Mark:20 30 40 50 60
- ▶ Jogh:25 15 25 15 25 15
- ▶ Peter:30
- ▶ David:20 30 10 30 20 10 20 30
- ▶ Julie:19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1

Output for Sample Input

- ▶ 710
- ▶ 2

Solution – Piggy bank

```
sum = 0
c = 0
while True:
    try:
        line = input().split(':')
        money = line[1].split(' ')
        twenty = True
        for i in range(len(money)):
            sum += int(money[i])
            if int(money[i]) < 20:
                twenty = False
        if twenty == True:
            c += 1

    except EOFError:
        break
print(sum)
print(c)
```


Homework 3 – Reverse numbers

<https://progcont.hu/progcont/100348/?pid=201508>

- ▶ Write a program that reads a sequence of numbers from the standard input and writes its elements in reverse order to the standard output.
- ▶ Each line of the input contains at least one integer, and the consecutive numbers are separated by exactly one space. The first number (n) in each line is the number of elements in the number line to be processed.
The program must write the following sequence of n integers in reverse order to the standard output. The end of the input is indicated by a line containing only one 0.
The program need not process this line.
- ▶ Make sure that the numbers written in a line are separated by exactly one space, and do not print spaces at the beginning and end of the line.

Homework 3 – Reverse numbers

<https://progcont.hu/progcont/100348/?pid=201508>

Sample Input

- ▶ 4 1 2 3 4
- ▶ 5 2 5 3 4 1
- ▶ 2 2 3
- ▶ 0

Output for Sample Input

- ▶ 4 3 2 1
- ▶ 1 4 3 5 2
- ▶ 3 2

Solution 1 – Reverse numbers

```
line = input().split()

while True:
    n = int(line[0])
    if n == 0:
        break
    i = n
    while i > 1:
        print(int(line[i]), end=' ')
        i -= 1
    print(int(line[i]))

    line = input().split()
```

Solution 2 – Reverse numbers

```
line = input().split()
while line[0] != '0':
    rev = line[:0:-1]
    revstr = ' '.join(x for x in rev)
    print(revstr)
    line = input().split()
```

Solution 3 – Reverse numbers

```
line = input().split()
while line[0] != '0':
    reversenum = list(reversed(line[1:]))
    print(*reversenum) #unpacking the list
    line = input().split()
```

Subroutines

A **subroutine** is a separate unit of an algorithm — a reusable, self-contained block of code. They serve two key purposes: **reuse** and **abstraction**.

- ▶ **Reuse:** The same sequence of instructions doesn't need to be written multiple times; it can be encapsulated into a single callable block. An example is embedding.
- ▶ **Abstraction:** A general concept can be represented by a reusable pattern with parameters that specify the details. A representation of generalization.

Subroutines

There are two main types of subroutines:

- ▶ **Function:** set of instructions to produce a value (the return value) somewhere in the code.
`return` instruction is included in the code, which returns the value given afterwards.
E.g. What is the cosine of 45°?
- ▶ `x = math.cos(45)`
- ▶ **Procedure:** a set of instructions that performs an action at a given point in the code (**no return value!!!**)
E.g. Write your name!
`print("David")`

Subroutines – Python

► Functions

- Built-in functions: `len()` – length of string
- They are grouped into modules – E.g. `sqrt()`
 - `import math`
 - `math.sqrt(x)` – square root of x
- You can also **define your own functions**

► Procedures

- (They have no return value)
- Also called a function in Python

Functions – Python

```
def function_name(parameters) :  
    statements
```

- ▶ The **function name** can be any identifier, the same rule applies as for variable names (it cannot be a Python keyword).
- ▶ The statements inside the function body must be **indented**.

Functions – Python

- ▶ The **header** (the first line) starts with the **def** keyword, followed by the function name, followed by the formal parameter list between round brackets, and finally a colon.
- ▶ The **formal parameter** list can be empty, or multiple parameters separated by commas.
- ▶ The round brackets are always required.
- ▶ The **function body** contains instructions that perform the function's task.

Functions without parameters

Example

```
def table7() -> None:
    n = 1
    while n < 11:
        print(n * 7, end=" ")
        n = n + 1
```

```
table7()
```

Functions without parameters

Example:

```
def table(base: int) -> None:
    n = 1
    while n < 11:
        print(n * base, end=" ")
        n = n + 1

table(13)
```

Multiple parameter function

Example:

```
def table(base: int, start: int, end: int) -> None:
    print("Part of the multiplication table of {}".format(base))
    n = start
    while n <= end:
        print("{} x {} = {}".format(n, base, n * base))
        n = n + 1
```

```
table(2, 1, 10)
table(8, 10, 20)
```

Local and global variables

- ▶ If you define variables in a function body, only the function itself has access to those variables.
- ▶ We say that these variables are **local variables** for the function.
- ▶ This is the case, for example, for the variables `base`, `start`, `end` and `n` in the previous exercise.
- ▶ Variables defined outside a function are **global variables**.
- ▶ These can be read from within a function, but **cannot be modified** unless explicitly declared as `global`.

Global variables

- ▶ Within a function, the **global** statement allows you to indicate inside a function definition which variables should be treated as global variables.

```
def increase()-> None:
    global a
    a = a + 1
    print(a)
```

```
a = 15
increase()
```

„Real” Functions

- ▶ The return statement defines the value that a function sends back to the caller. It marks the end of the function’s execution and returns the result to where the function was called. Example:

```
def cube(a: int) -> int:  
    return a * a * a
```

```
b = cube(9)  
print(b)
```


Optional parameters

- ▶ When defining Python functions, you can specify so-called **optional parameters**.
- ▶ This is used when there is a parameter to the function that is important to be a parameter because it is changed from time to time, but in the vast majority of cases it will have the same value.
- ▶ In this case, we can give the parameter a default value, and if the function does not receive that parameter when the function is called, it will use the default value.

Optional parameters

► Example

```
def quadrilateral(a: int, b: int = 0) -> int:  
    if b == 0:  
        b = a  
    return a * b
```

```
print('Area of square: ', quadrilateral(10))  
print('Area of rectangle: ', quadrilateral(10, 12))
```

Optional parameters

- ▶ Parameter passing by position

```
print('Area of square: ', quadrilateral(10))  
print('Area of rectangle: ', quadrilateral(10, 12))
```

- ▶ Parameter passing by name

```
print('Area of square: ', quadrilateral(a=10))  
print('Area of rectangle: ', quadrilateral(b=2, a=12))  
#the order of the parameters is reversible
```

Optional parameters

- ▶ Only **optional parameters** may be omitted when calling a function.
- ▶ **Required parameters** must always be specified (by position or by name).
- ▶ When mixing the two:
 - Positional arguments must come **first**, followed by named arguments.

Exercise

- ▶ Write a function that greets the user in a specific language.
By default, it should greet in **Hungarian**, but it can use other languages based on a **country code**.
- ▶ If the country code is not recognized, print an appropriate message.

Solution

```
def great(name: str, country_code: str = "HU"):
    match country_code:
        case "HU":
            print("Szia {}".format(name))
        case "EN":
            print("Hi {}".format(name))
        case "FI":
            print("Moi {}".format(name))
        case _:
            print("Country code not found!")
```

Parameter passing types

Python supports several **parameter passing concepts**:

- ▶ **By value** (full isolation): a copy of the original variable is created; changes do not affect the original.
- ▶ **By reference** (full linkage): the function operates on the original variable; any change affects it directly.
- ▶ **Python's model: *object-reference based***
 - The variable and the parameter reference the same object, but are different names.
 - You can modify the object (if it's mutable), but not reassign the original variable to another object.

Example

```
def modify(func):  
    func[0] = 2  # modify the object  
    func = [3]  # rebinds the local variable  
  
original = [1]  
modify(original)  
print(original)  # 1=Value 2=Object-reference 3=Reference
```


Main function

- ▶ Every Python program can define an **entry point** using the special variable `__name__`.
- ▶ When a Python file is executed directly, `__name__` is set to `"__main__"`.
- ▶ This allows you to write code that only runs when the file is executed — not when it is imported.

```
def main():  
    pass
```

```
if __name__ == "__main__":  
    main()
```

Examples

```
def add(a: int, b: int) -> int:  
    return a + b
```

```
def sum(n: int) -> int:  
    # 1+2+3+...+n  
    s = 0  
    for i in range(1, n + 1):  
        s = s + i  
    return s
```

```
def factorial(n: int) -> int:  
    #  $n*(n-1)*(n-2)*...*3*2*1$   
    f = 1  
    for i in range(n, 0, -1):  
        f = f * i  
    return f
```

```
def power(x: int, n: int) -> int:  
    # return  $x ** n$   
    p = 1  
    for i in range(n):  
        p = p * x  
    return p
```

```
def main():  
    print(add(5, 7))  
    print('Sum=', sum(5))  
    print('5!=', factorial(5))  
    print('2^10=', power(2, 10))
```

```
if __name__ == '__main__':  
    main()
```

Exercise

- ▶ Write functions that:
 - Fill a list with **10 random integers** between 1 and 100
 - Return the **minimum value**
 - Return the **index of the maximum value**
 - Return the **number of even elements**
 - Check whether a number is **prime** (True/False)
 - Check whether a number is **prime**

```
import random
import math
```

```
def generate(li: list) -> None:
    for i in range(10):
        li.append(random.randint(1, 100))
```

```
def minimum_value(li: list) -> int:
    min = li[0]
    for i in range(len(li)):
        if li[i] < min:
            min = li[i]
    return min
```

```
def maximum_index(li: list) -> int:
    maxi = 0
    for i in range(len(li)):
        if li[i] > li[maxi]:
            maxi = i
    return maxi
```

```
def evens(li: list) -> int:
    even = 0
    for i in range(len(li)):
        if li[i] % 2 == 0:
            even += 1
    return even
```

```
def isprim(n: int) -> bool:
    if n == 0 or n == 1:
        return False
    d = 2
    while d <= math.sqrt(n):
        if n % d == 0:
            return False
        d += 1
    return True
```

```
def prime_numbers(li: list) -> int:
    prime = 0
    for i in range(len(li)):
        if isprim(li[i]):
            prime += 1
    return prime
```

```
def main():
    li = []
    generate(li)
    print(li)
    print("Minimum value = ", minimum_value(li))
    print("Maximum index = ", maximum_index(li))
    print("Number of evens = ", evens(li))
    print("Number of primes = ", prime_numbers(li))
```

```
if __name__ == "__main__":
    main()
```

Homework 1

<https://viskillz.inf.unideb.hu/prog/#/?week=P1041>

- ▶ Write a function named *count_of_local_minimums()* that returns the number of elements in the list that are smaller than both of its neighbours.
- ▶ In case of the *main()* function, read lists elements until the given condition, and call the *count_of_local_minimums()* function and print out the number of local minimums.
 - Reads lists until EOF
 - Reads n test cases (lists)
 - Reads until an empty line

Homework 1

<https://viskillz.inf.unideb.hu/prog/#/?week=P1041>

Sample Input

```
97 -52 -12 41 87 19 35 -45 43
18 92 -21 -86 65
-60 -5 -73 -96 -31 1 72
-42 2 -67 -1 -35 46 32 -46 71 -56
-65 -63 3 -60 -14 61 41 -53
-60 84 47 -76 -68 4
```

Output for Sample Input

```
3
1
1
3
1
1
```

Homework 2

<https://viskillz.inf.unideb.hu/prog/#/?week=P1042>

- ▶ Write a function called *is_palindrome()* that returns whether the list given as parameter is a palindrome.
- ▶ In case of the *main()* function, read lists elements until the given condition, and call the *is_palindrome()* function and print out a True or a False value, if the list elements are palindroms or not.
 - Reads lists until EOF
 - Reads n test cases (lists)
 - Reads until an empty line

Homework 2

<https://viskillz.inf.unideb.hu/prog/#/?week=P1042>

Sample Input

1 2 3 4 5 6

6 5 4 3 2 1

1 2 3 2 1

1 1 1 1

1

-1

1 2 3 5 8 13

Output for Sample Input

False

False

True

True

True

True

False

Homework 3

<https://viskillz.inf.unideb.hu/prog/#/?week=P1052>

- ▶ Write a function called *double_even_digits()* that returns the string returned by duplicating the even digits of the string given as a parameter.
- ▶ In case of the *main()* function, read strings until the given condition, and call the *double_even_digits()* function and print out the new string.
 - Reads strings until EOF
 - Reads n test cases
 - Reads until an empty string

Homework 3

<https://viskillz.inf.unideb.hu/prog/#/?week=P1052>

Sample Input

```
0123456789  
a b c d e f g h  
aB1cD2eF3gH4
```

Output for Sample Input

```
001223445667889  
a b c d e f g h  
aB1cD22eF3gH44
```